



Université Abdelmalek Essaâdi
Faculté des sciences et techniques de Tanger



MST : SITBD | Module : Cloud et Edge Computing

RAPPORT DE PROJET

Déploiement d'une Infrastructure Cloud avec OpenStack, Terraform et Supervision SLA

Réalisé Par :
Nadir Mounim

Encadré Par :
Prof. C. El AMRANI

2025 - 2026

Table des Matières

Table des Matières	2
Table de figures	3
Introduction	4
Partie 1 : Déploiement et Utilisation d'OpenStack.....	5
1.1 Environnement Matériel et Logiciel	5
1.2 Installation de DevStack	6
1.2.1 Création de l'utilisateur stack.....	6
1.2.2 Clonage du dépôt DevStack.....	7
1.2.3 Configuration du fichier local.conf	7
1.2.4 Exécution du script d'installation	7
1.3 Vérification des Services OpenStack	8
1.4 Interface Horizon	9
1.5 Test de Nova et Neutron	10
1.5.1 Verification via CLI.....	10
1.5.2 Topologie réseau.....	11
1.6 Création d'une Instance IaaS avec CirrOS	12
1.6.1 Configuration des règles de sécurité	12
1.6.2 Démarrage et accès a l'instance CirrOS	13
1.7 Développement et Déploiement d'une Application SaaS.....	14
1.7.1 Architecture de l'application.....	14
1.7.2 Méthode de déploiement - Golden Image	14
1.7.3 Fonctionnalités de l'application.....	15
1.7.4 Vérification du déploiement	17
1.8 Images Disponibles dans Glance	18
Partie 2 : Infrastructure as Code avec Terraform	19
2.1 Présentation de Terraform	19
2.2 Configuration du Fichier main.tf	19
2.3 Exécution de Terraform	20
2.3.1 Initialisation	20
2.3.2 Application de la configuration	21
2.4 Vérification de la Création	22
2.5 Avantages de l'Approche IaC	23
Partie 3 : SLA et Supervision	24
3.1 Notion de SLA en Cloud Computing.....	24
3.2 Fichier SLA (sla.json)	24
3.3 Programme de Supervision (monitor.py)	25
3.4 Exécution et Rapport de Supervision	26
3.5 Automatisation avec Cron	27
Conclusion	28
Références	29

Table de figures

Figure 1— Paramtres memoire VirtualBox : 12288 Mo alloues a la VM	5
Figure 2— Paramtres processeur VirtualBox : 4 vCPUs	6
Figure 3—Creation de l'utilisateur stack avec droits sudo	6
Figure 4—Clonage du depot officiel DevStack depuis OpenDev	7
Figure 5—Contenu du fichier de configuration local.conf	7
Figure 6—Installation DevStack terminee avec succes en 402 secondes	7
Figure 7—20 services DevStack actifs : Nova, Neutron, Glance, Keystone, Horizon...	8
Figure 8—Page de connexion Horizon (http://192.168.56.101)	9
Figure 9—Tableau de bord Horizon : resume des ressources cloud	9
Figure 10—Liste complete des instances via CLI Nova	10
Figure 11—Reseaux et routeurs actifs via CLI Neutron	10
Figure 12—Topologie reseau Horizon : public, shared, private	11
Figure 13—Regles du groupe de securite default dans Horizon	12
Figure 14—Console Horizon : instance CirrOS avec invite de connexion	13
Figure 15—Commande route add Windows pour acceder aux IPs flottantes	13
Figure 16—Connexion PuTTY SSH a l'instance	13
Figure 17—Page de connexion InventoryPro (admin / admin123)	15
Figure 18— Tableau de bord InventoryPro avec metriques et liste d'articles	15
Figure 19—Modal d'ajout d'article avec generation automatique du SKU	16
Figure 20—Added item	16
Figure 21—Test curl de l'API : reponse {status: ok}	17
Figure 22—Statut systemd : apache2, myapp, mysql — tous actifs	17
Figure 23—Catalogue Glance : cirros-0.6.3, CentOS-7, MiniSaaS-v1	18
Figure 24—Contenu du fichier main.tf dans le terminal	19
Figure 25—Sortie de terraform init : provider installe avec succes	20
Figure 26—Sortie complte de terraform apply : Apply complete! 1 added	21
Figure 27—Instance Terraform-CentOS-Automated Active dans Horizon	22
Figure 28—Vue complete des instances dans Horizon	22
Figure 29—Contenu du fichier sla.json	24
Figure 30—Code source complet de monitor.py	25
Figure 31—Sortie de python3 monitor.py avec rapport de disponibilite	26
Figure 32—Fichier sla_report.txt avec plusieurs entrees horodatees	26
Figure 33—Crontab : execution automatique toutes les 5 minutes	27

Introduction

Ce rapport présente la réalisation d'un projet de cloud computing basé sur trois parties complémentaires. L'objectif principal est de déployer une infrastructure cloud complète en utilisant OpenStack via DevStack, d'automatiser la création d'instances avec Terraform, et de mettre en place un système de supervision basé sur un contrat de niveau de service (SLA).

L'ensemble des installations a été réalisé localement sur un ordinateur personnel équipé d'un processeur Intel Core i7-12650H, avec 16 Go de RAM et sous le système d'exploitation Windows 11. Une machine virtuelle VirtualBox a été configurée avec 12 288 Mo de RAM et 4 processeurs virtuels pour héberger l'environnement OpenStack.

Le projet se divise en trois grandes parties : le déploiement et l'utilisation d'OpenStack avec le développement d'une application SaaS, l'automatisation de l'infrastructure avec Terraform, et enfin la mise en place d'un SLA avec un système de supervision automatique.

Partie 1 : Déploiement et Utilisation d'OpenStack

1.1 Environnement Matériel et Logiciel

L'infrastructure de base repose sur une machine virtuelle VirtualBox configurée avec les ressources suivantes :

Composant	Spécification
Processeur hôte	Intel Core i7-12650H
RAM hôte	16 Go
RAM VM	12 288 Mo
CPU VM	4 processeurs virtuels
OS VM	Ubuntu 20.04 LTS
Plateforme Cloud	DevStack
Réseau	Interface Host-Only : 192.168.56.101

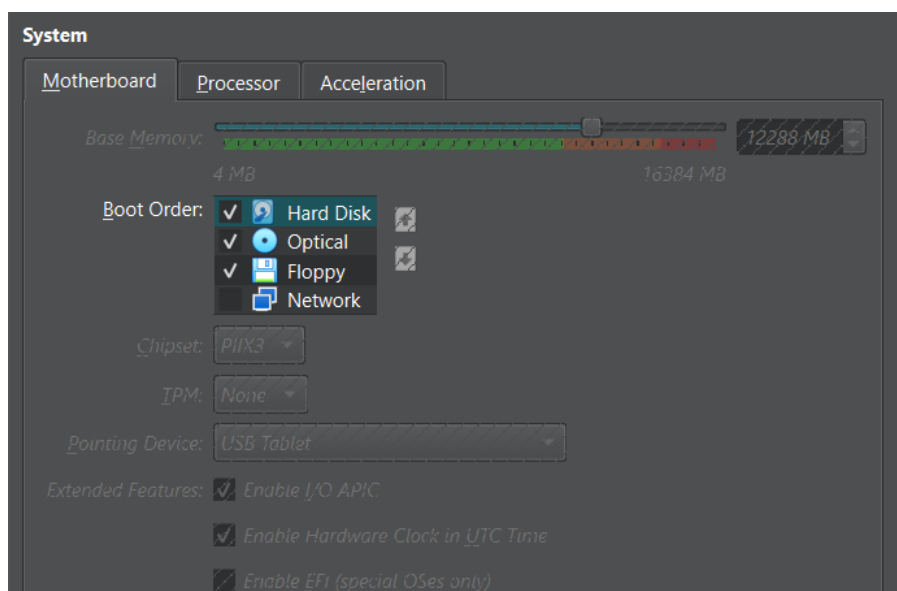


Figure 1— Paramètres mémoire VirtualBox : 12288 Mo alloués à la VM

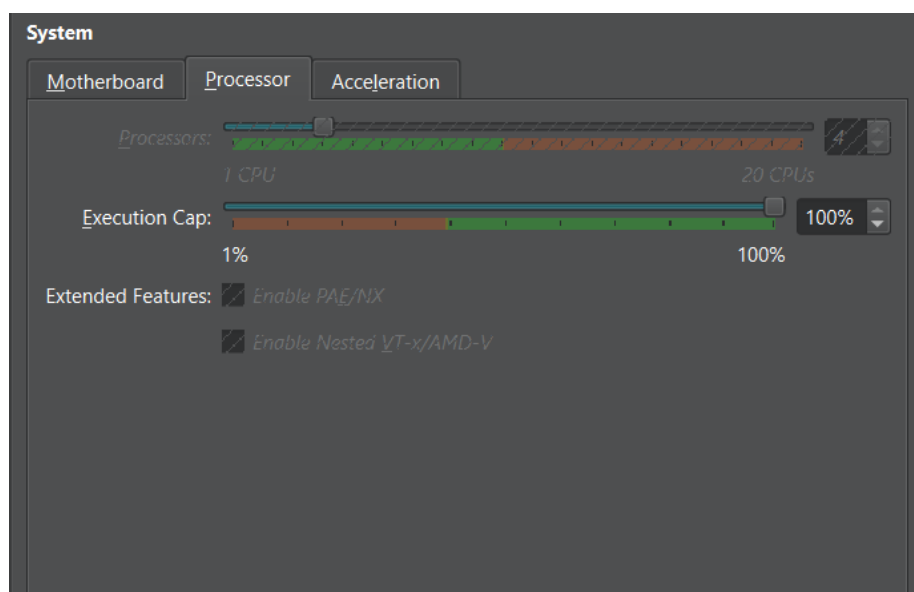


Figure 2— Paramètres processeur VirtualBox : 4 vCPUs

1.2 Installation de DevStack

L'installation de DevStack nécessite la création d'un utilisateur dédié nommé 'stack', puis le clonage du dépôt officiel DevStack depuis GitHub. Cette approche garantit une installation propre et reproductible de l'environnement OpenStack.

1.2.1 Création de l'utilisateur stack

La première étape consiste à créer un utilisateur système dédié pour l'exécution de DevStack :

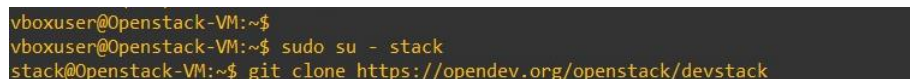
```
sudo useradd -s /bin/bash -d /opt/stack -m stack
sudo chmod +x /opt/stack
echo 'stack ALL=(ALL) NOPASSWD: ALL' | sudo tee /etc/sudoers.d/stack
```

```
vboxuser@Openstack-VM:~$
vboxuser@Openstack-VM:~$ sudo useradd -s /bin/bash -d /opt/stack -m stack
[sudo] password for vboxuser:
vboxuser@Openstack-VM:~$ echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
stack ALL=(ALL) NOPASSWD: ALL
```

Figure 3—Création de l'utilisateur stack avec droits sudo

1.2.2 Clonage du dépôt DevStack

```
su - stack
git clone https://opendev.org/openstack/devstack
cd devstack
```

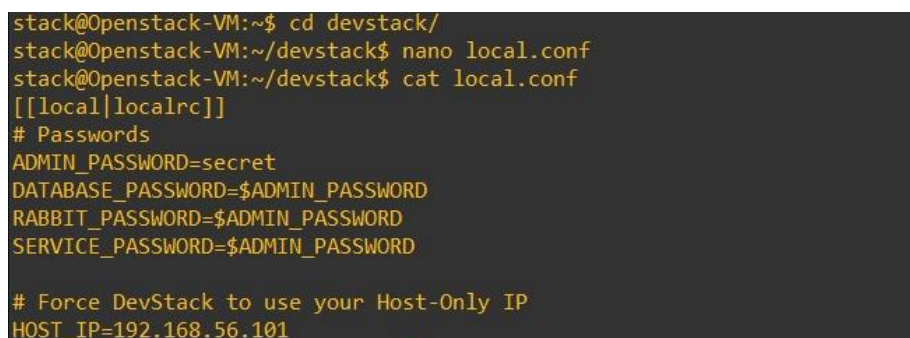


```
vboxuser@Openstack-VM:~$
vboxuser@Openstack-VM:~$ sudo su - stack
stack@Openstack-VM:~$ git clone https://opendev.org/openstack/devstack
```

Figure 4—Clonage du dépôt officiel DevStack depuis OpenDev

1.2.3 Configuration du fichier local.conf

Le fichier local.conf définit les paramètres essentiels de l'installation OpenStack, notamment les mots de passe des services et l'adresse IP de l'hôte :



```
stack@Openstack-VM:~$ cd devstack/
stack@Openstack-VM:~/devstack$ nano local.conf
stack@Openstack-VM:~/devstack$ cat local.conf
[[local|localrc]]
# Passwords
ADMIN_PASSWORD=secret
DATABASE_PASSWORD=$ADMIN_PASSWORD
RABBIT_PASSWORD=$ADMIN_PASSWORD
SERVICE_PASSWORD=$ADMIN_PASSWORD

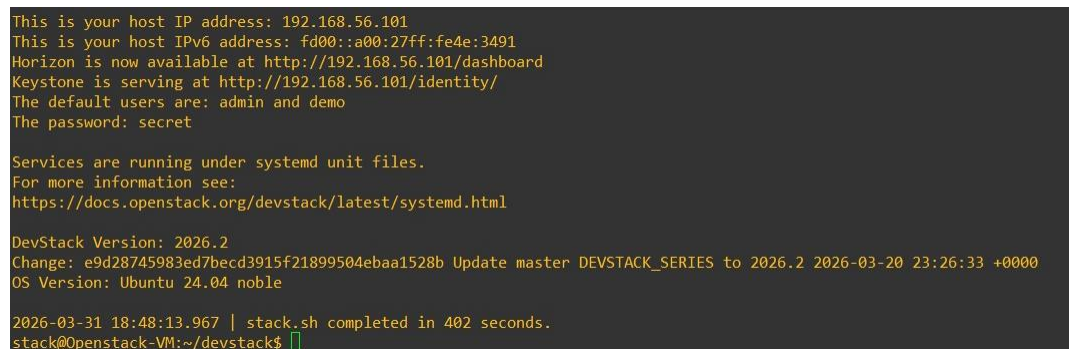
# Force DevStack to use your Host-Only IP
HOST_IP=192.168.56.101
```

Figure 5—Contenu du fichier de configuration local.conf

1.2.4 Exécution du script d'installation

L'exécution du script stack.sh lance l'installation complète de tous les composants OpenStack. Cette opération prend environ 30 à 60 minutes selon les ressources disponibles.

```
./stack.sh
```



```
This is your host IP address: 192.168.56.101
This is your host IPv6 address: fd00::a00:27ff:fe4e:3491
Horizon is now available at http://192.168.56.101/dashboard
Keystone is serving at http://192.168.56.101/identity/
The default users are: admin and demo
The password: secret

Services are running under systemd unit files.
For more information see:
https://docs.openstack.org/devstack/latest/systemd.html

DevStack Version: 2026.2
Change: e9d28745983ed7becd3915f21899504ebaa1528b Update master DEVSTACK_SERIES to 2026.2 2026-03-20 23:26:33 +0000
OS Version: Ubuntu 24.04 noble

2026-03-31 18:48:13.967 | stack.sh completed in 402 seconds.
stack@Openstack-VM:~/devstack$
```

Figure 6—Installation DevStack terminée avec succès en 402 secondes

1.3 Vérification des Services OpenStack

Après l'installation, tous les services DevStack doivent être actifs et fonctionnels. La commande suivante permet de vérifier l'état de chaque service :

```
sudo systemctl list-units 'devstack@*' --no-pager
```

Le résultat confirme que les 20 services sont chargés et en cours d'exécution, incluant Nova (compute), Neutron (réseau), Glance (images), Keystone (identité), Cinder (volumes) et Horizon (tableau de bord).

```
vboxuser@Openstack-VM:~$ sudo systemctl list-units 'devstack@*' --no-pager
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
devstack@c-api.service             loaded active running Devstack devstack@c-api.service
devstack@c-sch.service             loaded active running Devstack devstack@c-sch.service
devstack@c-vol.service             loaded active running Devstack devstack@c-vol.service
devstack@dstat.service             loaded active running Devstack devstack@dstat.service
devstack@etcd.service              loaded active running Devstack devstack@etcd.service
devstack@g-api.service             loaded active running Devstack devstack@g-api.service
devstack@keystone.service          loaded active running Devstack devstack@keystone.service
devstack@n-api-meta.service        loaded active running Devstack devstack@n-api-meta.service
devstack@n-api.service             loaded active running Devstack devstack@n-api.service
devstack@n-cond-cell1.service      loaded active running Devstack devstack@n-cond-cell1.service
devstack@n-cpu.service             loaded active running Devstack devstack@n-cpu.service
devstack@n-novnc-cell1.service     loaded active running Devstack devstack@n-novnc-cell1.service
devstack@n-sch.service             loaded active running Devstack devstack@n-sch.service
devstack@n-super-cond.service      loaded active running Devstack devstack@n-super-cond.service
devstack@neutron-api.service       loaded active running Devstack devstack@neutron-api.service
devstack@neutron-ovn-maintenance-worker.service loaded active running Devstack devstack@neutron-ovn-maintenance-worker.service
devstack@neutron-periodic-workers.service loaded active running Devstack devstack@neutron-periodic-workers.service
devstack@neutron-rpc-server.service loaded active running Devstack devstack@neutron-rpc-server.service
devstack@placement-api.service     loaded active running Devstack devstack@placement-api.service
devstack@q-ovn-agent.service       loaded active running Devstack devstack@q-ovn-agent.service

Legend: LOAD    → Reflects whether the unit definition was properly loaded.
          ACTIVE → The high-level unit activation state, i.e. generalization of SUB.
          SUB    → The low-level unit activation state, values depend on unit type.

20 loaded units listed. Pass --all to see loaded but inactive units, too.
To show all installed unit files use 'systemctl list-unit-files'.
vboxuser@Openstack-VM:~$
```

Figure 7—20 services DevStack actifs : Nova, Neutron, Glance, Keystone, Horizon...

1.4 Interface Horizon

Horizon est l'interface web d'OpenStack accessible via un navigateur. Elle permet de gérer l'ensemble des ressources cloud de manière graphique et intuitive.

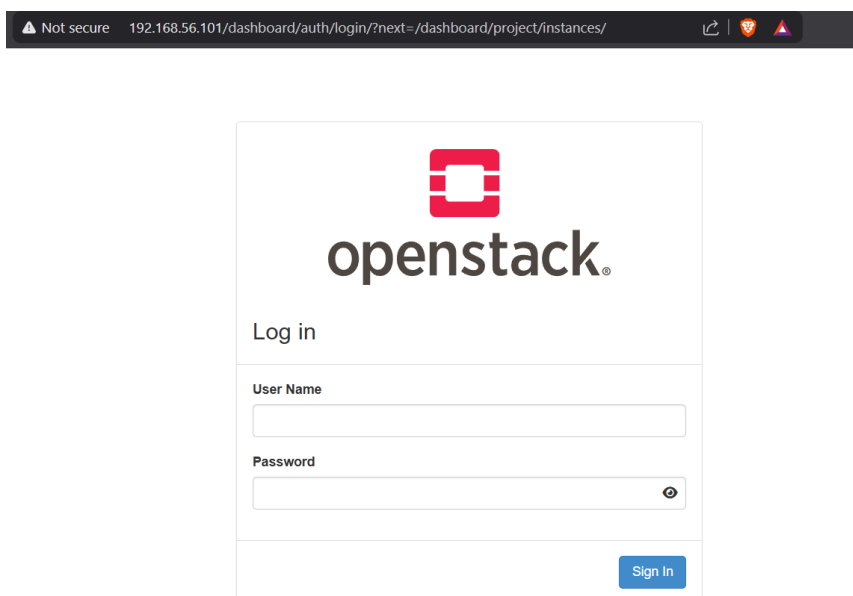


Figure 8—Page de connexion Horizon (<http://192.168.56.101>)

Après connexion avec les identifiants administrateur (admin/secret), le tableau de bord principal affiche un résumé des ressources utilisées : instances, vCPUs, RAM, réseaux, groupes de sécurité et adresses IP flottantes.

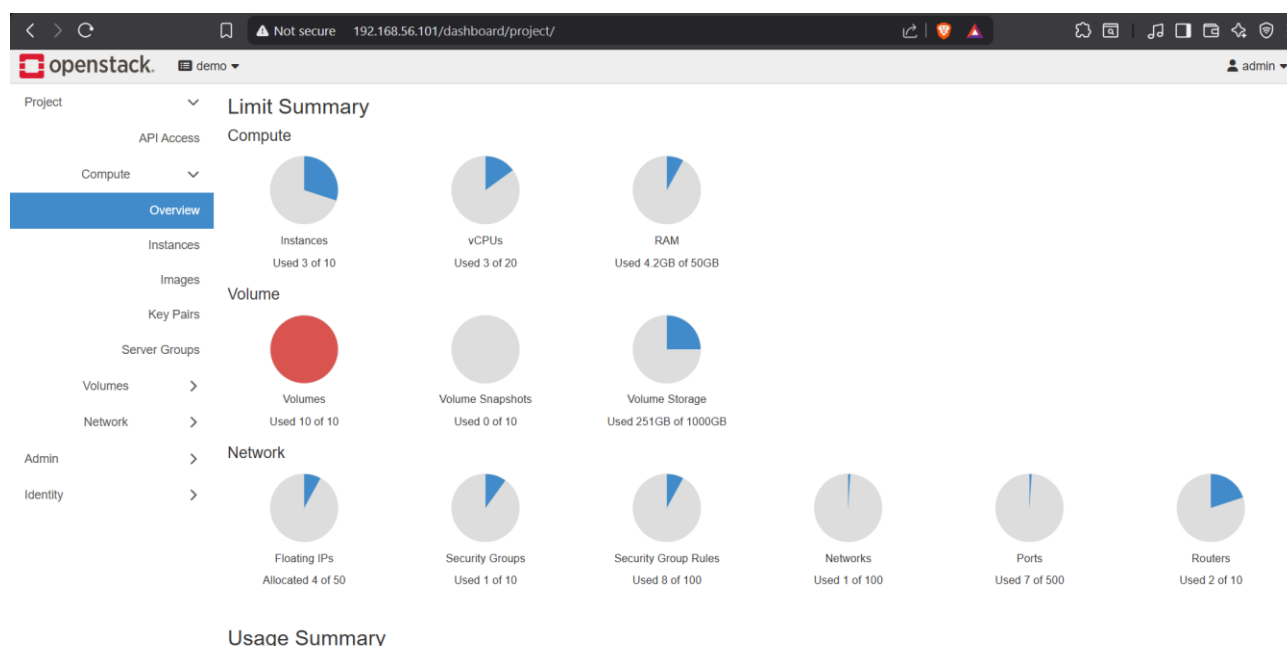


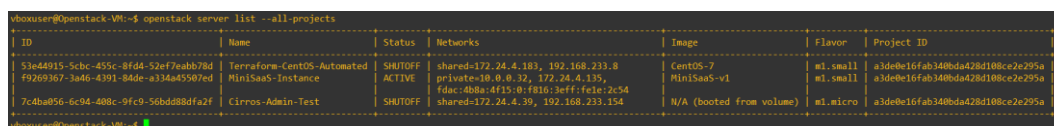
Figure 9—Tableau de bord Horizon : résumé des ressources cloud

1.5 Test de Nova et Neutron

Nova est le service de calcul d'OpenStack responsable de la gestion des instances. Neutron est le service réseau qui gère les réseaux virtuels, sous-réseaux et routeurs.

1.5.1 Verification via CLI

```
source /opt/stack/devstack/openrc admin admin
openstack server list --all-projects
```

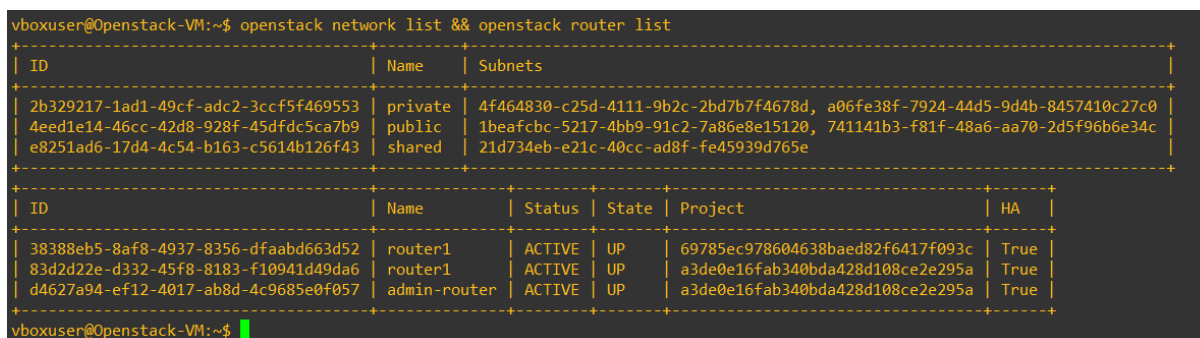


```
vboxuser@Openstack-VM:~$ openstack server list --all-projects
```

ID	Name	Status	Networks	Image	Flavor	Project ID
53e44915-5c8c-455c-8fd4-52ef7eabb78d	Terraform-CentOS-Automated	SHUTOFF	shared=172.24.4.183, 192.168.233.8	CentOS-7	m1.small	a3de0e16fab340bda428d108ce2e295a
f9269367-3a46-4391-84de-a334a4507ed	MiniSaaS-Instance	ACTIVE	private=10.0.0.32, 172.24.4.135, fdac:4b8a:4f15:0:f810:3eff:fe1e:2c54	MiniSaaS-v1	m1.small	a3de0e16fab340bda428d108ce2e295a
7c4ba956-6c94-408c-9fc9-56bd88df2f2f	Cirros-Admin-Test	SHUTOFF	shared=172.24.4.30, 192.168.233.154	N/A (booted from volume)	m1.micro	a3de0e16fab340bda428d108ce2e295a

Figure 10—Liste complète des instances via CLI Nova

```
openstack network list && openstack router list
```



```
vboxuser@Openstack-VM:~$ openstack network list && openstack router list
```

ID	Name	Subnets
2b329217-1ad1-49cf-adc2-3ccf5f469553	private	4f464830-c25d-4111-9b2c-2bd7b7f4678d, a06fe38f-7924-44d5-9d4b-8457410c27c0
4eed1e14-46cc-42d8-928f-45dfdc5ca7b9	public	1beafc9c-5217-4bb9-91c2-7a86e8e15120, 741141b3-f81f-48a6-aa70-2d5f96b6e34c
e8251ad6-17d4-4c54-b163-c5614b126f43	shared	21d734eb-e21c-40cc-ad8f-fe45939d765e

ID	Name	Status	State	Project	HA
38388eb5-8af8-4937-8356-dfaabd663d52	router1	ACTIVE	UP	69785ec978604638baed82f6417f093c	True
83d2d22e-d332-45f8-8183-f10941d49da6	router1	ACTIVE	UP	a3de0e16fab340bda428d108ce2e295a	True
d4627a94-ef12-4017-ab8d-4c9685e0f057	admin-router	ACTIVE	UP	a3de0e16fab340bda428d108ce2e295a	True

Figure 11—Réseaux et routeurs actifs via CLI Neutron

1.5.2 Topologie réseau

La topologie réseau d'OpenStack comprend trois réseaux principaux : le réseau public (172.24.4.0/24) pour les adresses IP flottantes, le réseau privé (10.0.0.0/26) pour les instances Ubuntu, et le réseau partagé (192.168.233.0/24) pour les instances Cirros et CentOS.

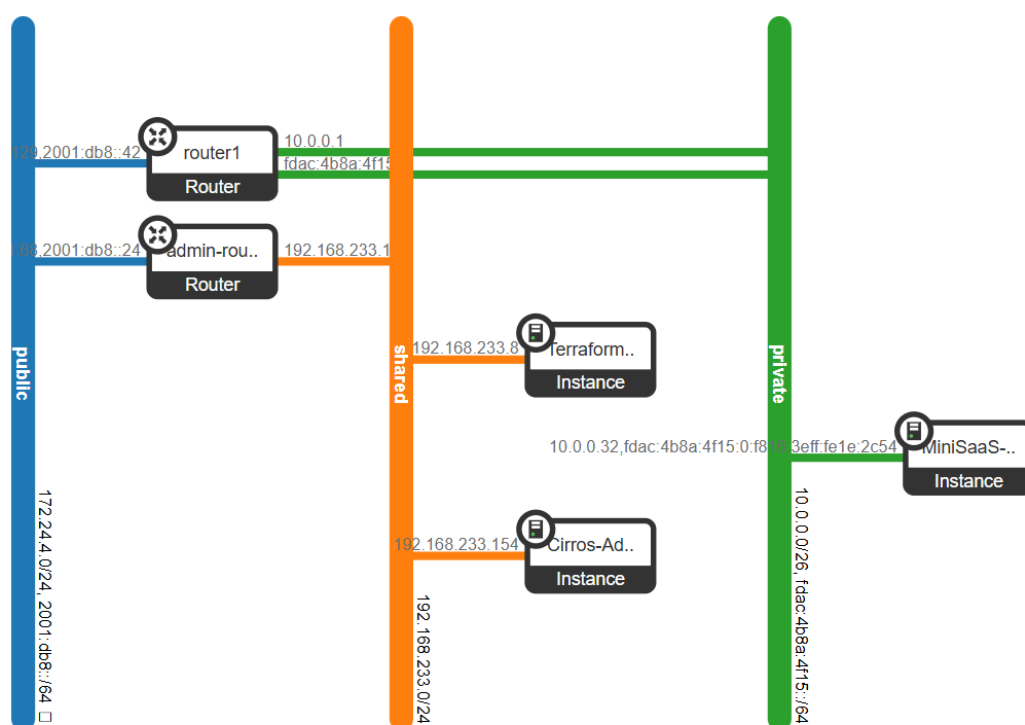


Figure 12—Topologie réseau Horizon : public, shared, private

1.6 Création d'une Instance IaaS avec CirrOS

CirrOS est une image Linux minimaliste utilisée pour tester les déploiements cloud. Elle sert à valider le bon fonctionnement du service Nova et de la connectivité réseau.

1.6.1 Configuration des règles de sécurité

Les règles de sécurité définissent les flux réseau autorisés vers les instances. Les règles configurées autorisent le trafic SSH (port 22), HTTP (port 80), l'API Flask (port 5000) ainsi que les requêtes ICMP (ping) :

+ Add Rule

Delete Rules

Displaying 8 items

☐	Direction	Ether Type	IP Protocol	Port Range	Remote IP Prefix	Remote Security Group	Description	Actions
☐	Egress	IPv4	Any	Any	0.0.0.0/0	-	-	Edit Rule ▼
☐	Egress	IPv6	Any	Any	::/0	-	-	Edit Rule ▼
☐	Ingress	IPv4	Any	Any	-	default	-	Edit Rule ▼
☐	Ingress	IPv4	ICMP	Any	0.0.0.0/0	-	-	Edit Rule ▼
☐	Ingress	IPv4	TCP	22 (SSH)	0.0.0.0/0	-	-	Edit Rule ▼
☐	Ingress	IPv4	TCP	80 (HTTP)	0.0.0.0/0	-	-	Edit Rule ▼
☐	Ingress	IPv4	TCP	5000	0.0.0.0/0	-	-	Edit Rule ▼
☐	Ingress	IPv6	Any	Any	-	default	-	Edit Rule ▼

Displaying 8 items

Figure 13—Règles du groupe de sécurité default dans Horizon

1.6.2 Démarrage et accès à l'instance CirrOS

L'instance CirrOS a été créée avec le flavor m1.micro (256 Mo RAM) et connectée au réseau partage. L'accès se fait via la console Horizon ou via PuTTY avec la clé SSH admin-key.

```

further output written to /dev/ttyS0
[ 6.855603] virtio_blk virtio2: [vdal 2097152 512-byte logical blocks (1.07 G
0/1.00 GiB)
[ 6.899354] GPT:Primary header thinks Alt. header is not at the end of the di
sk.
[ 6.899884] GPT:229375 != 2097151
[ 6.900018] GPT:Alternate GPT header not at the end of the disk.
[ 6.900231] GPT:229375 != 2097151
[ 6.900523] GPT: Use GNU Parted to correct GPT errors.
[ 7.367583] virtio_gpu virtio0: [drm] drm_plane_enable_fb_damage_clips() not called
[ 7.483099] random: crng init done
[ 51.426128] clocksource: Long readout interval, skipping watchdog check: cs_nsec: 1203673308 wd_nsec: 1203672370

login as 'cirros' user. default password: 'gocubsgo'. use 'sudo' for root.
cirros login: cirros
Password:
$ ls
$ whoami
cirros
$ -

```

Figure 14—Console Horizon : instance CirrOS avec invite de connexion

```

C:\Windows\System32>route add 172.24.4.0 mask 255.255.255.0 192.168.56.101
OK!

C:\Windows\System32>

```

Figure 15—Commande route add Windows pour accéder aux IPs flottantes

```

login as: cirros
Authenticating with public key "imported-openssh-key"
$ ls
$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1442 qdisc pfifo_fast qlen 1000
    link/ether fa:16:3e:59:51:b5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.233.103/24 brd 192.168.233.255 scope global dynamic noprefixrou
te eth0
        valid_lft 42780sec preferred_lft 37380sec
    inet6 fe80::f816:3eff:fe59:51b5/64 scope link
        valid_lft forever preferred_lft forever
$ uname -a
Linux cirros 5.15.0-117-generic #127-Ubuntu SMP Fri Jul 5 20:13:28 UTC 2024 x86_
64 GNU/Linux
$
$ whoami
cirros

```

Figure 16—Connexion PuTTY SSH à l'instance

1.7 Développement et Déploiement d'une Application SaaS

L'application SaaS développée est InventoryPro, un système de gestion d'inventaire professionnel. Elle suit une architecture trois-tiers classique déployée sur une instance Ubuntu OpenStack.

1.7.1 Architecture de l'application

Couche	Technologie et Rôle
Frontend	HTML5 / CSS3 / JavaScript vanilla - Interface utilisateur responsive
Backend	Python Flask (port 5000) - API REST avec authentification par session
Base de données	MySQL 8.0 - Stockage persistant des données
Serveur web	Apache2 (port 80) - Proxy inverse vers Flask
Système	Ubuntu 20.04

1.7.2 Méthode de déploiement - Golden Image

Une approche 'Golden Image' a été adoptée : l'application est préinstallée directement dans l'image disque QCOW2 avant le déploiement. Cette méthode, utilisée dans l'industrie par les grands opérateurs cloud, garantit que chaque instance lancée est immédiatement opérationnelle sans configuration post-démarrage.

La technique utilisée combine guestmount (montage FUSE de l'image) et chroot (environnement isolé) pour installer les paquets directement dans l'image sans démarrer de machine virtuelle :

```
sudo guestmount -a focal-server-cloudimg-amd64.img -i --rw /mnt/guest
sudo mount --bind /dev /mnt/guest/dev
sudo mount --bind /proc /mnt/guest/proc
sudo chroot /mnt/guest /bin/bash -c "apt install -y apache2 mysql-server python3-pip"
pip3 install flask mysql-connector-python
```

L'image a ensuite été redimensionnée de 2,2 Go à 10 Go pour accommoder les installations, puis chargée dans Glance sous le nom 'MiniSaaS-v1'.

1.7.3 Fonctionnalités de l'application

InventoryPro propose les fonctionnalités suivantes :

- Authentification sécurisée par session Flask avec base de données MySQL
- Tableau de bord avec métriques en temps réel : total articles, stock disponible, valeur totale
- Gestion complète des articles : création, suppression, recherche et filtrage
- Génération automatique des codes SKU (ITEM-001, ITEM-002, etc.)
- Filtrage par statut (En stock, Stock faible, Rupture) et par catégorie
- API REST protégée par authentification (GET/POST/DELETE /api/items)

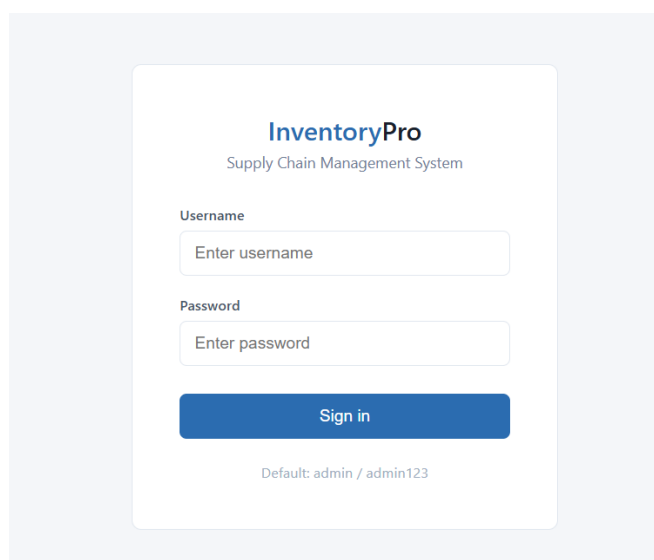


Figure 17—Page de connexion InventoryPro (admin / admin123)

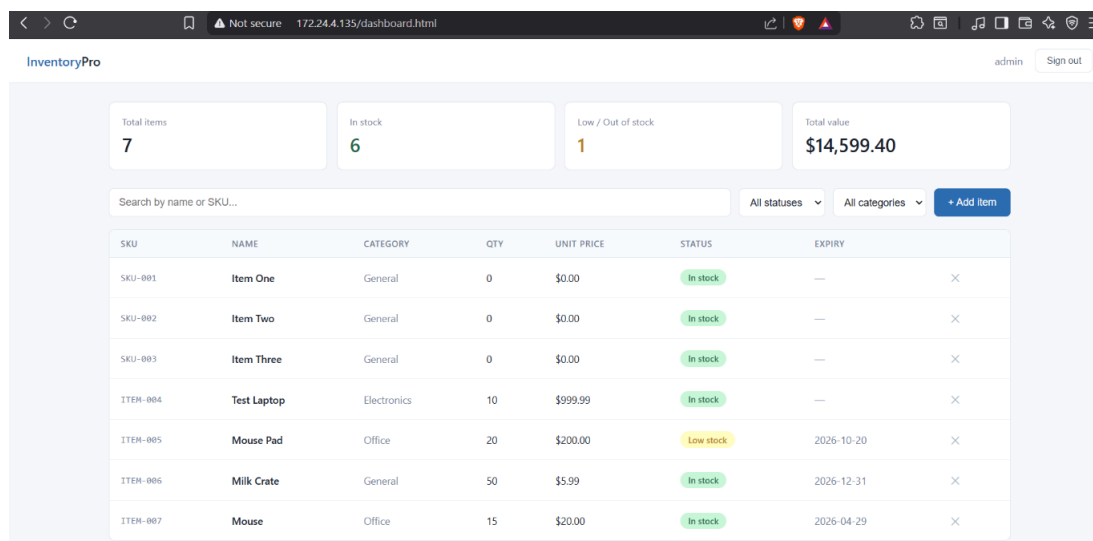


Figure 18—Tableau de bord InventoryPro avec métriques et liste d'articles

Add inventory item

Category

Office

Product name

Mouse

Quantity

15

Unit price (\$)

20

Status

In stock

Expiry date

04/29/2026

Cancel

Save item

Figure 19—Modal d'ajout d'article avec génération automatique du SKU

SKU	NAME	CATEGORY	QTY	UNIT PRICE	STATUS	EXPIRY	
SKU-001	Item One	General	0	\$0.00	In stock	—	×
SKU-002	Item Two	General	0	\$0.00	In stock	—	×
SKU-003	Item Three	General	0	\$0.00	In stock	—	×
ITEM-004	Test Laptop	Electronics	10	\$999.99	In stock	—	×
ITEM-005	Mouse Pad	Office	20	\$200.00	Low stock	2026-10-20	×
ITEM-006	Milk Crate	General	50	\$5.99	In stock	2026-12-31	×
ITEM-007	Mouse	Office	15	\$20.00	In stock	2026-04-29	×

Figure 20—Added item

1.7.4 Vérification du déploiement

L'application est accessible via l'adresse IP flottante 172.24.4.135. Les services sont vérifiés opérationnels :

```
vboxuser@Openstack-VM:~$ curl http://172.24.4.135/api/health
{"status":"ok"}
vboxuser@Openstack-VM:~$
```

Figure 21—Test curl de l'API : réponse {status: ok}

```
ubuntu@minisaas-instance:~$ systemctl status apache2 myapp mysql --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-04-11 00:58:24 UTC; 17min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 568 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
  Main PID: 719 (apache2)
    Tasks: 55 (limit: 2325)
   Memory: 20.0M
   CGroup: /system.slice/apache2.service
           └─719 /usr/sbin/apache2 -k start
             └─723 /usr/sbin/apache2 -k start
               └─726 /usr/sbin/apache2 -k start

Apr 11 00:57:17 minisaas-instance systemd[1]: Starting The Apache HTTP Server...
Apr 11 00:58:18 minisaas-instance apachectl[666]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using fdac:4b8a:4f15:0:f816:3eff:fe1e:2c54. Set the 'Listen this message
Apr 11 00:58:24 minisaas-instance systemd[1]: Started The Apache HTTP Server.

● myapp.service - My Flask API
   Loaded: loaded (/etc/systemd/system/myapp.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-04-11 01:00:58 UTC; 15min ago
  Main PID: 990 (python3)
    Tasks: 1 (limit: 2325)
   Memory: 23.3M
   CGroup: /system.slice/myapp.service
           └─990 /usr/bin/python3 /opt/myapp/app.py

Apr 11 01:01:24 minisaas-instance python3[990]: * Debug mode: off
Apr 11 01:01:31 minisaas-instance python3[990]: WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
Apr 11 01:01:31 minisaas-instance python3[990]: * Running on all addresses (0.0.0.0)
Apr 11 01:01:31 minisaas-instance python3[990]: * Running on http://127.0.0.1:5000
Apr 11 01:01:31 minisaas-instance python3[990]: * Running on http://10.0.0.32:5000
Apr 11 01:01:32 minisaas-instance python3[990]: Press CTRL+C to quit
Apr 11 01:02:14 minisaas-instance python3[990]: 127.0.0.1 - - [11/Apr/2026 01:02:14] "GET /api/health HTTP/1.1" 200 -
Apr 11 01:07:53 minisaas-instance python3[990]: 127.0.0.1 - - [11/Apr/2026 01:07:53] "POST /api/login HTTP/1.1" 200 -
Apr 11 01:07:54 minisaas-instance python3[990]: 127.0.0.1 - - [11/Apr/2026 01:07:54] "GET /api/items HTTP/1.1" 200 -
Apr 11 01:14:22 minisaas-instance python3[990]: 127.0.0.1 - - [11/Apr/2026 01:14:22] "GET /api/health HTTP/1.1" 200 -

● mysql.service - MySQL Community Server
   Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-04-11 01:00:58 UTC; 15min ago
   Process: 579 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
  Main PID: 728 (mysqld)
    Status: "Server is operational"
    Tasks: 38 (limit: 2325)
   Memory: 406.2M
   CGroup: /system.slice/mysql.service
           └─728 /usr/sbin/mysqld
```

Figure 22—Statut systemd : apache2, myapp, mysql — tous actifs

1.8 Images Disponibles dans Glance

Le catalogue d'images OpenStack contient trois images actives et publiques :

- cirros-0.6.3-x86_64-disk (20,69 Mo) - Image de test minimaliste
- CentOS-7 (861,06 Mo) - Image utilisée par Terraform
- MiniSaaS-v1 (2,33 Go) - Image Ubuntu avec application InventoryPro préinstallée

Images

Click here for filters or full text search.									+ Create Image	Delete Images
Displaying 3 Items										
☐	Owner	Name	Type	Status	Visibility	Protected	Disk Format	Size		
☐	> demo	CentOS-7	Image	Active	Public	No	QCOW2	861.06 MB	Launch	▼
☐	> admin	cirros-0.6.3-x86_64-disk	Image	Active	Public	No	QCOW2	20.69 MB	Launch	▼
☐	> admin	MiniSaaS-v1	Image	Active	Public	No	QCOW2	2.33 GB	Launch	▼
Displaying 3 Items										

Figure 23—Catalogue Glance : cirros-0.6.3, CentOS-7, MiniSaaS-v1

Partie 2 : Infrastructure as Code avec Terraform

2.1 Présentation de Terraform

Terraform est un outil d'Infrastructure as Code (IaC) développé par HashiCorp. Il permet de définir, planifier et créer des infrastructures cloud de manière déclarative via des fichiers de configuration. Contrairement aux approches impératives, Terraform décrit l'état final désiré de l'infrastructure et détermine automatiquement les actions nécessaires pour l'atteindre.

La version 1.7.5 de Terraform a été installée sur la machine virtuelle OpenStack, avec le fournisseur terraform-provider-openstack en version 1.53.0.

2.2 Configuration du Fichier main.tf

Le fichier de configuration Terraform définit deux éléments essentiels : le fournisseur OpenStack avec les paramètres d'authentification, et la ressource représentant la machine virtuelle CentOS à créer.

```
main.tf terraform.tfstate terraform.tfstate.backup
terraform {
  required_providers {
    openstack = {
      source = "terraform-provider-openstack/openstack"
      version = "~> 1.53.0"
    }
  }
}

provider "openstack" {
  user_name     = "admin"
  tenant_name   = "demo"
  password      = "secret"
  auth_url      = "http://192.168.56.101/identity/v3"
  domain_name   = "Default"
}

resource "openstack_compute_instance_v2" "centos_vm" {
  name           = "Terraform-CentOS-Automated"
  image_name     = "CentOS-7"
  flavor_name    = "m1.small"
  key_pair       = "admin-key"
  security_groups = ["default"]
  network {
    name = "shared"
  }
}
```

Figure 24—Contenu du fichier main.tf dans le terminal

La ressource `openstack_compute_instance_v2` définit une instance avec les paramètres suivants :

- Nom : Terraform-CentOS-Automated
- Image : CentOS-7
- Flavor : m1.small (2 Go RAM, 1 vCPU, 20 Go disque)
- Clé SSH : admin-key
- Groupe de sécurité : default
- Réseau : shared

2.3 Exécution de Terraform

2.3.1 Initialisation

La commande `terraform init` initialise le répertoire de travail, télécharge le fournisseur OpenStack et configure le backend de stockage de l'état :

```
vboxuser@Openstack-VM:~/terraform-projet$ terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of terraform-provider-openstack/openstack from the de
pendency lock file
- Using previously-installed terraform-provider-openstack/openstack v1.53.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
vboxuser@Openstack-VM:~/terraform-projet$ ls
main.tf  terraform.tfstate  terraform.tfstate.backup
vboxuser@Openstack-VM:~/terraform-projet$
```

Figure 25—Sortie de `terraform init` : provider installé avec succès

2.3.2 Application de la configuration

La commande `terraform apply -auto-approve` planifie et applique les changements sans demander de confirmation manuelle. Terraform crée l'instance CentOS en 24 secondes :

```
terraform apply -auto-approve
```

```
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# openstack_compute_instance_v2.centos_vm will be created
+ resource "openstack_compute_instance_v2" "centos_vm" {
  + access_ip_v4      = (known after apply)
  + access_ip_v6      = (known after apply)
  + all_metadata      = (known after apply)
  + all_tags          = (known after apply)
  + availability_zone  = (known after apply)
  + created           = (known after apply)
  + flavor_id         = (known after apply)
  + flavor_name       = "m1.small"
  + force_delete      = false
  + id               = (known after apply)
  + image_id          = (known after apply)
  + image_name        = "CentOS-7"
  + key_pair          = "admin-key"
  + name              = "Terraform-CentOS-Automated"
  + power_state       = "active"
  + region            = (known after apply)
  + security_groups   = [
    + "default",
  ]
  + stop_before_destroy = false
  + updated           = (known after apply)

  + network {
    + access_network = false
    + fixed_ip_v4    = (known after apply)
    + fixed_ip_v6    = (known after apply)
    + floating_ip    = (known after apply)
    + mac            = (known after apply)
    + name           = "private"
    + port           = (known after apply)
    + uuid           = (known after apply)
  }
}

Plan: 1 to add, 0 to change, 0 to destroy.
openstack_compute_instance_v2.centos_vm: Creating...
openstack_compute_instance_v2.centos_vm: Still creating... [10s elapsed]
openstack_compute_instance_v2.centos_vm: Still creating... [20s elapsed]
openstack_compute_instance_v2.centos_vm: Creation complete after 24s [id=82265a02-cf5d-43a2-a42c-d8018a6c5a44]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
vboxuser@Openstack-VM:~/terraform-projet$
```

Figure 26—Sortie complète de `terraform apply` : *Apply complete! 1 added*

2.4 Vérification de la Création

Après l'exécution, l'instance Terraform-CentOS-Automated est visible dans Horizon avec le statut Active. Elle est connectée au réseau shared avec l'adresse IP 192.168.233.8.

Instances

Instance ID =

Filter

Launch Instance

Delete Instances

More Actions

Displaying 4 Items

☐	Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions
☐	Terraform-CentOS-Automated	CentOS-7	192.168.233.8	m1.small	admin-key	Active	nova	None	Running	2 minutes	Create Snapshot
☐	MiniSaaS-Instance	MiniSaaS-v1	10.0.0.32, 172.24.4.135, fdac:4b8a:4f15:0:f816:3eff:fe1e:2c54	m1.small	admin-key	Shutoff	nova	None	Shut Down	3 days, 22 hours	Start Instance
☐	SaaS-Web-Server-Final	Ubuntu-20.04	192.168.233.249, 172.24.4.98	m1.medium	admin-key	Shutoff	nova	None	Shut Down	1 week, 1 day	Start Instance
☐	Cirros-Admin-Test	cirros-0.6.3-x86_64-disk	192.168.233.154, 172.24.4.39	m1.micro	admin-key	Active	nova	None	Running	1 week, 3 days	Create Snapshot

Displaying 4 Items

Figure 27—Instance Terraform-CentOS-Automated Active dans Horizon

La liste complète des instances confirme la coexistence de l'instance Terraform avec les autres instances du projet :

Instances

Instance ID =

Filter

Launch Instance

Delete Instances

More Actions

Displaying 3 Items

☐	Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions
☐	Terraform-CentOS-Automated	CentOS-7	192.168.233.8, 172.24.4.183	m1.small	admin-key	Shutoff	nova	None	Shut Down	3 hours, 12 minutes	Start Instance
☐	MiniSaaS-Instance	MiniSaaS-v1	10.0.0.32, 172.24.4.135, fdac:4b8a:4f15:0:f816:3eff:fe1e:2c54	m1.small	admin-key	Active	nova	None	Running	4 days, 1 hour	Create Snapshot
☐	Cirros-Admin-Test	cirros-0.6.3-x86_64-disk	192.168.233.154, 172.24.4.39	m1.micro	admin-key	Shutoff	nova	None	Shut Down	1 week, 3 days	Start Instance

Displaying 3 Items

Figure 28—Vue complète des instances dans Horizon

2.5 Avantages de l'Approche IaC

L'utilisation de Terraform pour la création d'infrastructure présente plusieurs avantages par rapport aux approches manuelles :

- Reproductibilité : la même configuration produit toujours le même résultat
- Versioning : le fichier main.tf peut être versionné avec Git
- Idempotence : terraform apply peut être exécuté plusieurs fois sans effet de bord
- Documentation : le code Terraform sert de documentation vivante de l'infrastructure
- Automatisation : intégration possible dans des pipelines CI/CD

Partie 3 : SLA et Supervision

3.1 Notion de SLA en Cloud Computing

Un Service Level Agreement (SLA) est un contrat formel entre un fournisseur de services cloud et ses clients qui définit les niveaux de service garantis. Dans le contexte d'OpenStack, le SLA porte principalement sur la disponibilité des instances de calcul, mesurée comme le ratio entre le temps de fonctionnement effectif et le temps total évalué.

L'objectif de disponibilité fixé dans ce projet est de 99,5% sur une période d'évaluation journalière. Cela signifie qu'une instance peut être indisponible au maximum 7 minutes et 12 secondes par jour tout en respectant le SLA.

3.2 Fichier SLA (sla.json)

Le contrat SLA est représenté par un fichier JSON qui définit le service surveillé et l'objectif de disponibilité :

```
vboxuser@Openstack-VM:~/terraform-projet$ cat ~/sla.json
{
  "service": "OpenStack Compute Instances",
  "target_availability": 99.5
}
vboxuser@Openstack-VM:~/terraform-projet$
```

Figure 29—Contenu du fichier sla.json

3.3 Programme de Supervision (monitor.py)

Le programme de supervision est développé en Python et utilise le SDK officiel OpenStack pour interagir avec les APIs. Il suit une logique en cinq étapes :

1. Chargement de l'objectif SLA depuis le fichier sla.json
2. Authentification auprès de Keystone (service d'identité OpenStack)
3. Interrogation de Nova (service de calcul) pour obtenir la liste de toutes les instances
4. Calcul du taux de disponibilité : (instances actives / total instances) x 100
5. Génération et enregistrement du rapport de disponibilité

Le programme utilise le paramètre `all_projects=True` pour surveiller les instances de tous les projets (admin et démo), conformément à une supervision globale de l'infrastructure.

```
import openstack
import json
import datetime

# 1. Load the SLA target
with open('/home/vboxuser/sla.json', 'r') as f:
    sla_data = json.load(f)
target_availability = sla_data['target_availability']

# 2. Authenticate with the OpenStack API
conn = openstack.connect(
    auth_url='http://192.168.56.101/identity/v3',
    project_name='admin',
    username='admin',
    password='secret',
    user_domain_name='Default',
    project_domain_name='Default'
)

# 3. Query the Nova Compute API for all instances
total_instances = 0
active_instances = 0

# all_projects=True ensures we see VMs in 'demo' and 'admin'
for server in conn.compute.servers(all_projects=True):
    total_instances += 1
    if server.status == 'ACTIVE':
        active_instances += 1

# 4. Calculate Availability
if total_instances > 0:
    current_availability = (active_instances / total_instances) * 100
else:
    current_availability = 0.0

# 5. Generate the SLA Report
timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
report = f"[{timestamp}] SLA Monitoring Report\n"
report += f"Total Instances Found: {total_instances}\n"
report += f"Active Instances: {active_instances}\n"
report += f"Current Availability: {current_availability:.2f}%\n"
report += f"Target SLA: {target_availability}%\n"

if current_availability >= target_availability:
    report += "STATUS: SLA RESPECTED (Pass)\n"
else:
    report += "STATUS: SLA VIOLATED (Fail)\n"
report += "-" * 40 + "\n"

# Print to screen and append to a log file
print(report)
with open('/home/vboxuser/sla_report.txt', 'a') as log_file:
    log_file.write(report)
vboxuser@Openstack-VM:~/terraform-projet$
```

Figure 30—Code source complet de monitor.py

3.4 Exécution et Rapport de Supervision

L'exécution du programme génère un rapport horodate indiquant le nombre total d'instances trouvées, le nombre d'instances actives, le taux de disponibilité calculé, et le statut du SLA (RESPECTED ou VIOLATED) :

```
vboxuser@Openstack-VM:~$ python3 ~/monitor.py
[2026-04-14 00:25:40] SLA Monitoring Report
Total Instances Found: 3
Active Instances: 2
Current Availability: 66.67%
Target SLA: 99.5%
STATUS: SLA VIOLATED (Fail)
-----
```

Figure 31—Sortie de python3 monitor.py avec rapport de disponibilité

Le fichier sla_report.txt accumule l'historique de toutes les exécutions, permettant une analyse de la disponibilité dans le temps :

```
vboxuser@Openstack-VM:~$ cat ~/sla_report.txt | tail -20
Total Instances Found: 3
Active Instances: 2
Current Availability: 66.67%
Target SLA: 99.5%
STATUS: SLA VIOLATED (Fail)
-----
[2026-04-14 00:35:03] SLA Monitoring Report
Total Instances Found: 3
Active Instances: 2
Current Availability: 66.67%
Target SLA: 99.5%
STATUS: SLA VIOLATED (Fail)
-----
[2026-04-14 00:40:15] SLA Monitoring Report
Total Instances Found: 3
Active Instances: 2
Current Availability: 66.67%
Target SLA: 99.5%
STATUS: SLA VIOLATED (Fail)
-----
```

Figure 32—Fichier sla_report.txt avec plusieurs entrées horodatées

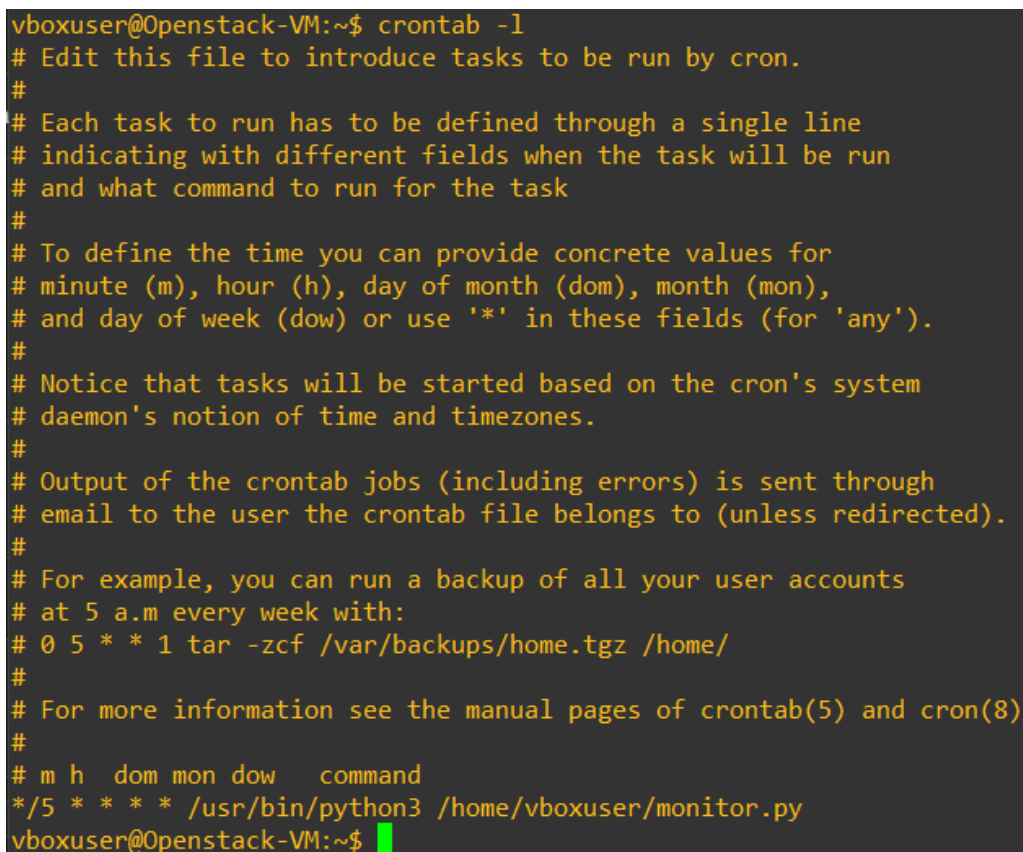
Le rapport indique un taux de disponibilité de 66.67% lors de certaines mesures (2 instances actives sur 3 total). Cela s'explique par la présence d'instances en état SHUTOFF dans l'inventaire. Ce résultat démontre que le système de supervision fonctionne correctement et détecte fidèlement les violations du SLA - ce qui est précisément l'objectif du système de surveillance.

3.5 Automatisation avec Cron

Pour assurer une surveillance continue, le script `monitor.py` est exécuté automatiquement toutes les 5 minutes via le planificateur de tâches cron. La configuration suivante a été ajoutée au crontab de l'utilisateur `vboxuser` :

```
*/5 * * * * /usr/bin/python3 /home/vboxuser/monitor.py
```

Cette expression cron signifie : exécuter la commande toutes les 5 minutes (`*/5`), pour chaque heure (`*`), chaque jour du mois (`*`), chaque mois (`*`) et chaque jour de la semaine (`*`).



```
vboxuser@Openstack-VM:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
*/5 * * * * /usr/bin/python3 /home/vboxuser/monitor.py
vboxuser@Openstack-VM:~$
```

Figure 33—Crontab : exécution automatique toutes les 5 minutes

Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux du cloud computing à travers trois axes complémentaires. La mise en place d'OpenStack via DevStack a fourni une plateforme IaaS complète et fonctionnelle, permettant de gérer des instances virtuelles, des réseaux et des images de manière similaire aux grands fournisseurs cloud commerciaux.

Le développement et le déploiement de l'application InventoryPro illustre une approche moderne de déploiement SaaS en utilisant la technique des Golden Images. Cette méthode, inspirée des pratiques industrielles, garantit la reproductibilité des déploiements et facilite la mise à l'échelle horizontale.

L'utilisation de Terraform pour l'Infrastructure as Code démontre l'importance de l'automatisation dans les environnements cloud modernes. La définition déclarative de l'infrastructure en code source permet un contrôle précis, une traçabilité complète et une reproductibilité parfaite des déploiements.

Enfin, le système de supervision SLA avec monitor.py et cron établit une base solide pour la gestion des niveaux de service. La détection automatique des violations du SLA permet une réaction rapide aux problèmes de disponibilité.

L'ensemble de ce projet constitue une introduction complète et pratique aux technologies de virtualisation appliquées au cloud computing, avec des cas d'usage concrets et des outils utilisés dans l'industrie.

Références

- OpenStack Documentation - <https://docs.openstack.org>
- DevStack Documentation - <https://docs.openstack.org/devstack>
- Terraform OpenStack Provider - <https://registry.terraform.io/providers/terraform-provider-openstack/openstack>
- Flask Documentation - <https://flask.palletsprojects.com>
- MySQL 8.0 Documentation - <https://dev.mysql.com/doc/refman/8.0/en>
- OpenStack Python SDK - <https://docs.openstack.org/openstacksdk>
- Apache HTTP Server Documentation - <https://httpd.apache.org/docs>